Project 1 of Neural Network and Deep Learning: Reconstructing MLP and CNN on MNIST

Yuheng Li
Student ID: 23307130334
https://github.com/yuheng-li-ai/CV_HW2

Abstract

This report studies handwritten digit classification on MNIST by reconstructing a multilayer perceptron (MLP) and a simple convolutional neural network (CNN) from NumPy-based components. The implementation includes the forward and backward propagation of the linear layer, the softmax cross-entropy loss, a manual two-dimensional convolution operator, model training, validation, testing, and visualization. The MLP baseline obtains a test accuracy of 0.8724, while the simple CNN improves the test accuracy to 0.9467 under a comparable training setting. Additional experiments are conducted for momentum optimization and early stopping. Momentum improves the MLP test accuracy to 0.9389 and the CNN test accuracy to 0.9758, giving the best result in this project. Early stopping does not activate within the five-epoch training setting and therefore does not change the final performance. Further robustness and visualization analyses show that the CNN is more stable than the MLP under Gaussian input noise and learns more spatially meaningful features.

1 Introduction

MNIST classification is a standard task for studying neural network models on image data. In this project, the goal is not only to obtain high accuracy, but also to reconstruct important neural network components manually. Therefore, the implementation avoids using deep learning modules that directly solve the required operations. The main components implemented in the code include linear-layer propagation, softmax cross-entropy loss, convolution, optimization, model training, validation, and evaluation.

The experiments are organized around two required baselines. First, an MLP is trained as a simple reference model. Second, a simple CNN is implemented and compared with the MLP under a similar training configuration. After the two baselines, additional directions are studied. The main selected directions are optimization with momentum and regularization with early stopping. Robustness analysis and visualization are also included to support the discussion of model behavior.

2 Method

2.1 MLP Baseline

The MLP treats each MNIST image as a flattened vector. A 28×28 image is reshaped into a 784-dimensional input. The model structure is

$$784 \to 600 \to 10, \tag{1}$$

with ReLU activation after the hidden linear layer. The final output dimension is 10, corresponding to the ten digit classes. The training uses softmax cross-entropy loss. The implemented linear layer computes the affine transformation in the forward pass and propagates gradients to both parameters and inputs in the backward pass.

The baseline MLP is trained with learning rate 0.06, batch size 32, and 5 epochs. A validation split of 10,000 samples is used from the training set. The model checkpoint with the best validation accuracy is saved during training.

2.2 Simple CNN

The CNN uses the original two-dimensional image structure instead of flattening the image at the input. The convolution operator is implemented manually in NumPy. The final CNN architecture is intentionally simple:

$$\begin{aligned} &\text{Conv2D} \to \text{ReLU} \to \text{Flatten} \\ &\to \text{Linear} \to \text{ReLU} \to \text{Linear}. \end{aligned} \tag{2}$$

The convolution layer uses 8 output channels and a kernel size of 3. The hidden linear layer has dimension 128. The model does not use bottleneck blocks, residual connections, or other advanced architectural designs. This keeps the comparison focused on the difference between a flattened MLP representation and a basic convolutional representation.

The CNN is trained with the same learning rate, batch size, and number of epochs as the MLP baseline: learning rate 0.06, batch size 32, and 5 epochs. This makes the MLP-vs-CNN comparison fairer because the main difference is the model structure.

2.3 Momentum Optimization

For the optimization direction, momentum gradient descent is implemented and tested on both MLP and CNN. Momentum

maintains a velocity term for each optimizable parameter. Instead of updating parameters only according to the current gradient, the optimizer combines the current gradient with the accumulated velocity. This can accelerate convergence and reduce oscillation during training.

The momentum coefficient is set to 0.9. Other training settings remain the same as the corresponding baseline unless explicitly stated. The purpose is to isolate the effect of the optimizer.

2.4 Early Stopping

For the regularization direction, early stopping is implemented in the training runner. The validation accuracy is monitored at epoch boundaries. If the validation accuracy does not improve for the specified patience, training can stop before reaching the maximum epoch number. In the final experiments, early stopping does not activate within the 5-epoch setting, so its result is identical to the corresponding baseline.

2.5 Robustness and Visualization

Robustness is evaluated by adding Gaussian noise to test images without retraining the models. This measures how the trained models behave under input perturbation. Error analysis and visualization include confusion matrices, misclassified samples, MLP first-layer weights, and CNN first-layer kernels.

3 Experimental Results

3.1 Baseline Comparison

Table 1 summarizes the main validation and test results. The MLP baseline reaches 0.86660 validation accuracy and 0.8724 test accuracy. The CNN baseline reaches 0.94430 validation accuracy and 0.9467 test accuracy. The CNN therefore clearly outperforms the MLP.

The result is consistent with the expected advantage of convolution for image data. The MLP ignores the spatial arrangement of pixels after flattening. In contrast, the CNN applies local filters over the image and shares parameters across spatial positions. This inductive bias makes the CNN more suitable for digit images.

3.2 Momentum Optimization

Momentum improves both models. For the MLP, the test accuracy increases from 0.8724 to 0.9389. For the CNN, the test accuracy increases from 0.9467 to 0.9758. The CNN with momentum gives the best overall result in this project.

The improvement indicates that plain gradient descent is not the best optimizer for these models under the chosen learning rate and epoch budget. Momentum helps the parameters move more consistently along useful descent directions.

3.3 Early Stopping

Early stopping is tested as the regularization direction. In both MLP and CNN experiments, the validation accuracy does not stop improving within the 5-epoch training setting. Therefore, early stopping does not activate and does not change the final test accuracy. This negative result is still useful because it shows that early stopping is not effective when the maximum epoch number is already small and the validation curve has not degraded.

3.4 Robustness to Gaussian Noise

Table 2 reports test accuracy under Gaussian noise. The same trained MLP and CNN checkpoints are evaluated on noisy test images, and no retraining is used.

Table 2: Robustness under Gaussian noise.

Noise σ	MLP Accuracy	CNN Accuracy
0.00	0.8724	0.9467
0.05	0.8621	0.9470
0.10	0.8212	0.9437
0.20	0.6789	0.9152
0.30	0.5434	0.8153

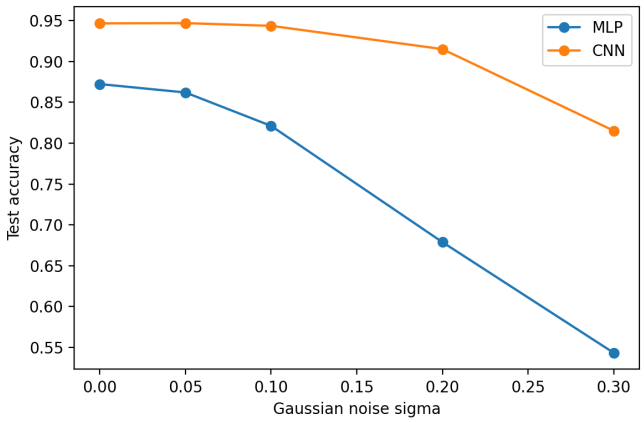


Figure 4: Accuracy under Gaussian input noise.

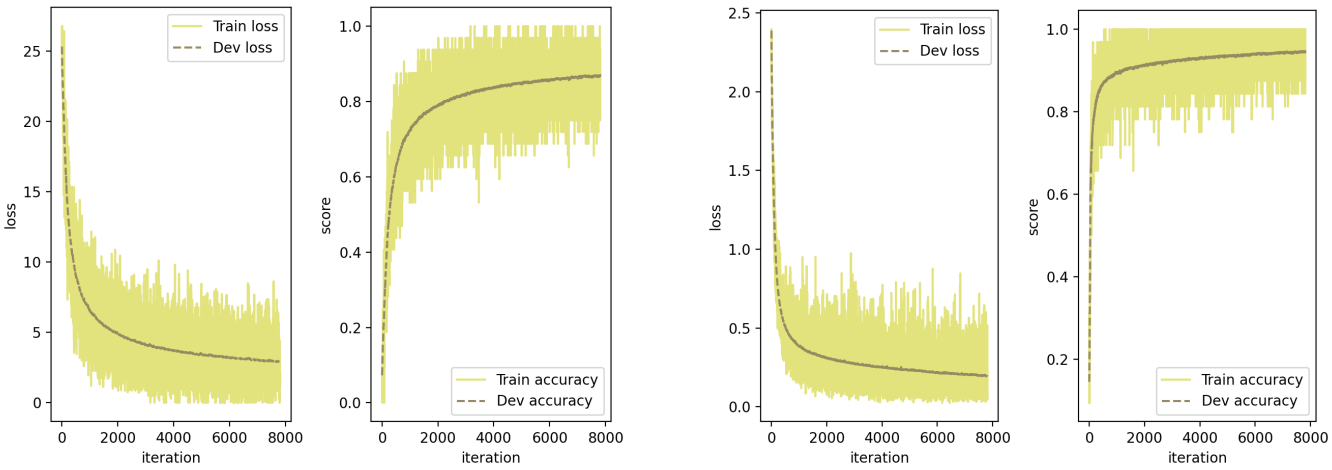
The CNN remains more stable as the noise level increases. At $\sigma = 0.30$, the MLP accuracy drops to 0.5434, while the CNN still reaches 0.8153. This suggests that the convolutional representation is more robust to local perturbations.

4 Visualization and Error Analysis

Confusion matrices and misclassified examples are used to examine the error patterns of the two models. The CNN makes fewer mistakes than the MLP, which matches the quantitative accuracy comparison. The visualized misclassified samples show that difficult cases often have ambiguous shapes or unusual writing styles.

Table 1: Main experimental results.

Experiment	Validation Accuracy	Test Accuracy	Main Observation
MLP baseline	0.86660	0.8724	Stable baseline, limited by flattened input
CNN baseline	0.94430	0.9467	CNN improves over MLP
MLP + Momentum	0.93550	0.9389	Momentum improves MLP optimization
CNN + Momentum	0.97520	0.9758	Best overall result
MLP + Early stopping	0.86660	0.8724	Early stopping does not activate
CNN + Early stopping	0.94430	0.9467	Early stopping does not activate



(a) MLP baseline learning curve.

(b) CNN baseline learning curve.

Figure 1: Learning curves of the two baseline models.

Learned representations are also visualized. The first-layer weights of the MLP can be reshaped into image-like patterns, but they are connected to all pixels and do not explicitly preserve locality. The CNN kernels are small spatial filters, so they directly capture local patterns such as strokes and edges.

5 Discussion

The experiments show that model structure has a strong influence on MNIST classification. The MLP baseline is able to learn useful decision boundaries, but flattening removes explicit spatial structure. The CNN uses local convolution and parameter sharing, which are better matched to image inputs. This explains why the CNN baseline improves the test accuracy from 0.8724 to 0.9467.

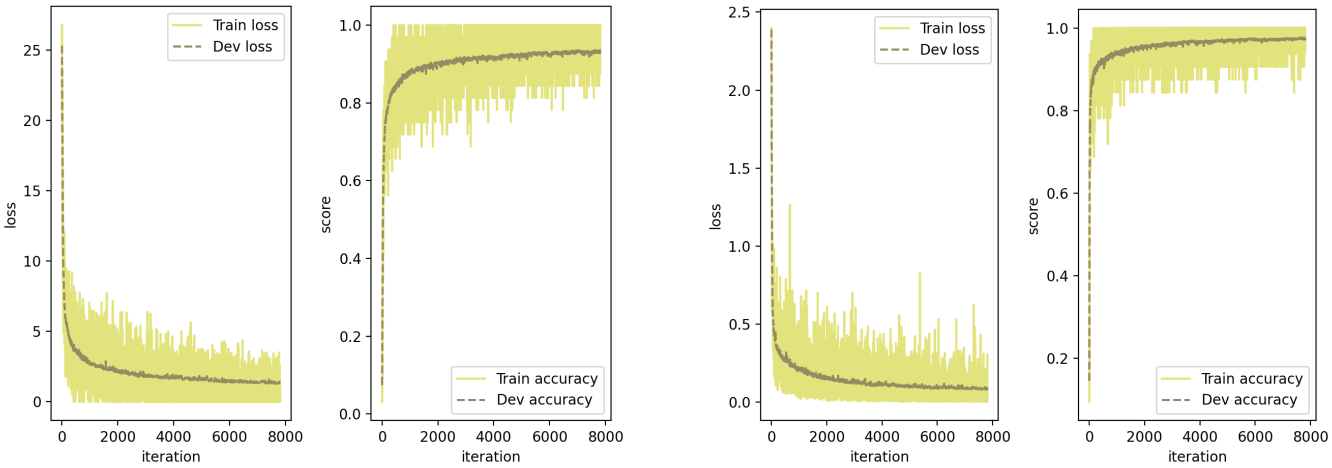
The optimization experiment shows that momentum is beneficial for both models. The best result is obtained by the CNN with momentum, reaching 0.9758 test accuracy. This result suggests that the CNN architecture provides a strong representation, while momentum further improves the training dynamics.

Early stopping gives a negative result in this setting. Since the experiments only train for 5 epochs and the validation accuracy does not degrade, the stopping condition does not activate. Therefore, early stopping does not improve performance here. This does not mean early stopping is useless in general; it only means that under this specific setting it is not the limiting factor.

The robustness results further support the advantage of CNNs. When Gaussian noise is added to the test images, both models degrade, but the CNN accuracy decreases more slowly. The visualization results also support the quantitative comparison: CNN kernels capture local image patterns, while the MLP relies on fully connected weights over flattened inputs.

6 Conclusion

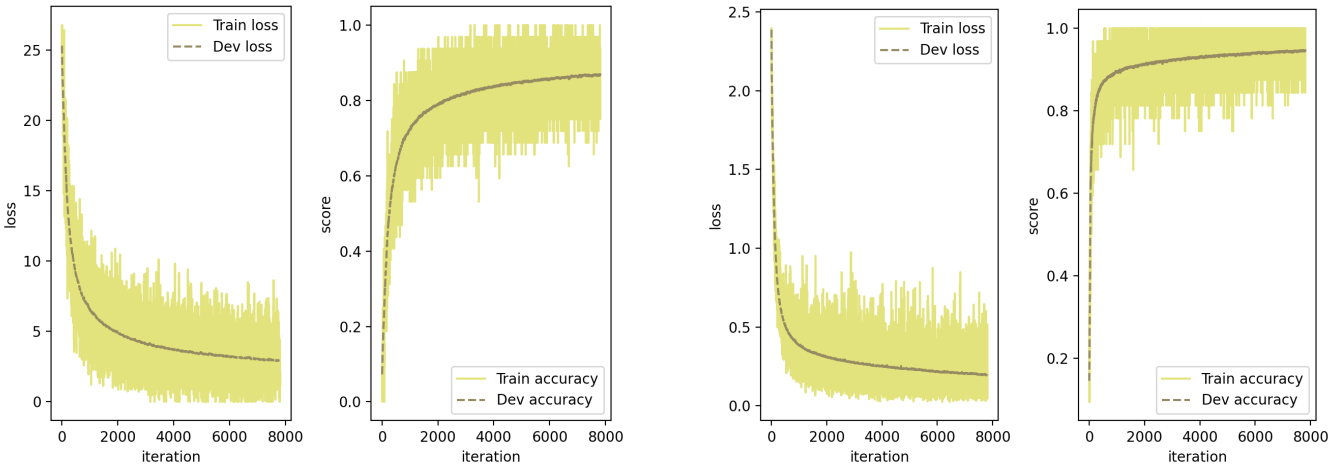
This project implements and evaluates an MLP and a simple CNN on MNIST. The required components, including linear-layer propagation, softmax cross-entropy, and manual convolution, are implemented in NumPy. The MLP baseline reaches 0.8724 test accuracy, while the simple CNN reaches 0.9467. Momentum optimization improves the MLP to 0.9389 and the CNN to 0.9758, which is the best result. Early stopping does not activate under the five-epoch setting, so it does not change performance. Robustness and visualization analyses show that the CNN learns more suitable and stable image features than the MLP.



(a) MLP with momentum.

(b) CNN with momentum.

Figure 2: Learning curves for the momentum experiments.



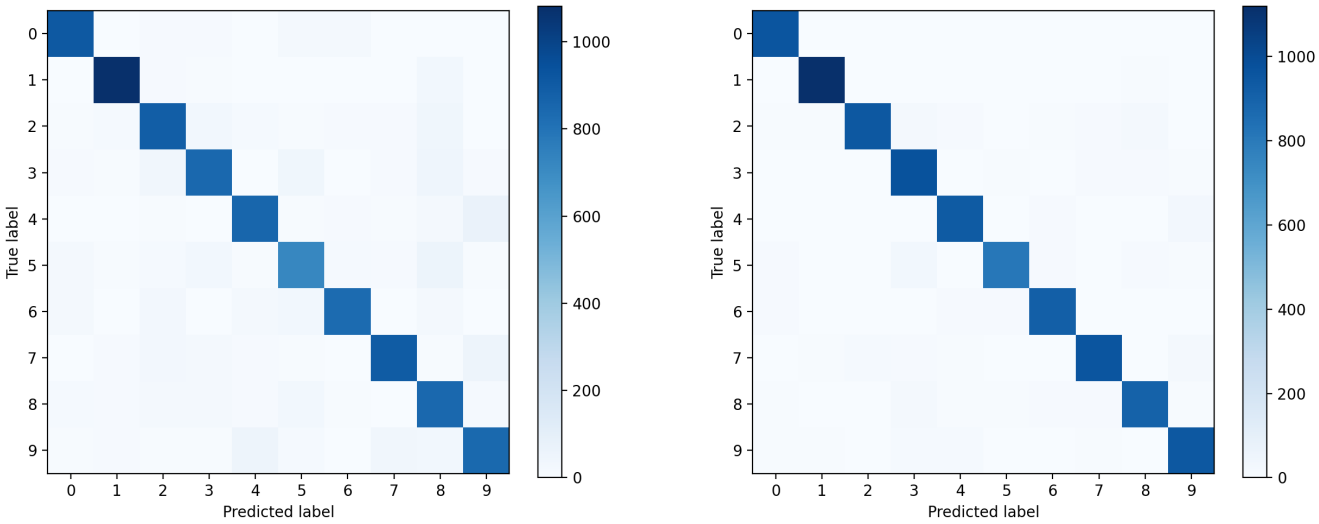
(a) MLP with early stopping.

(b) CNN with early stopping.

Figure 3: Learning curves for early stopping experiments.

Code and Checkpoints

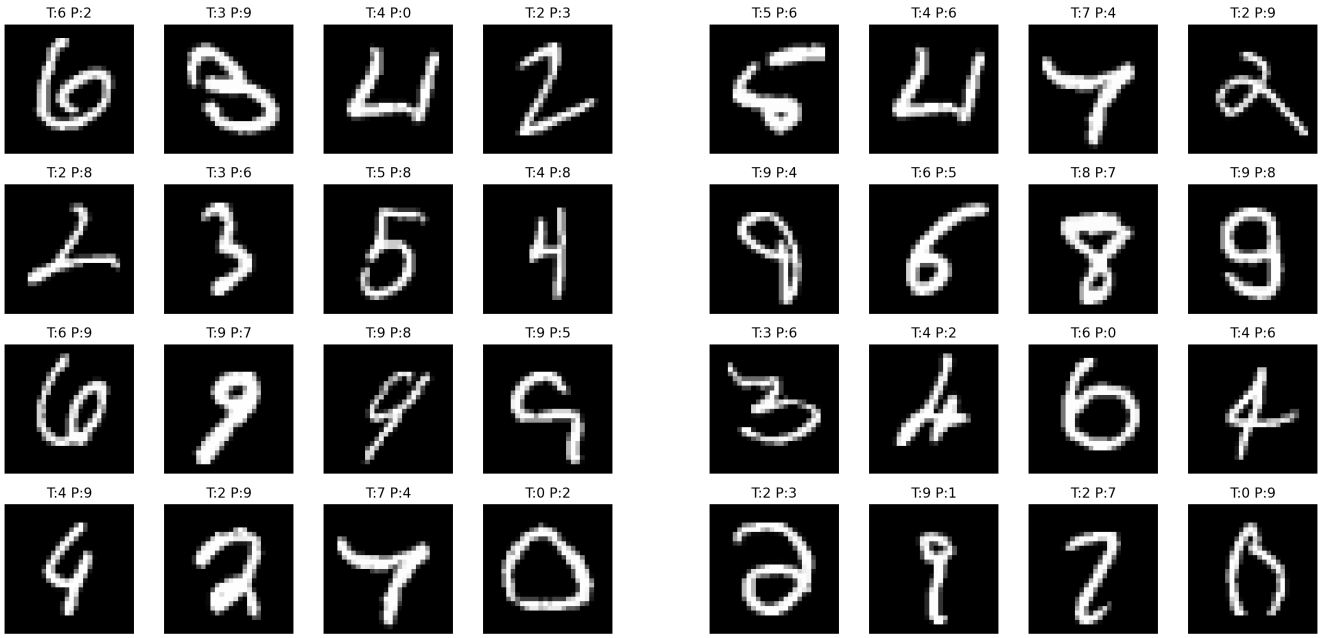
The code repository is available at https://github.com/yuheng-li-ai/CV_HW2. The checkpoint/model weights link should be added separately before final submission: **[To be added]**.



(a) MLP confusion matrix.

(b) CNN confusion matrix.

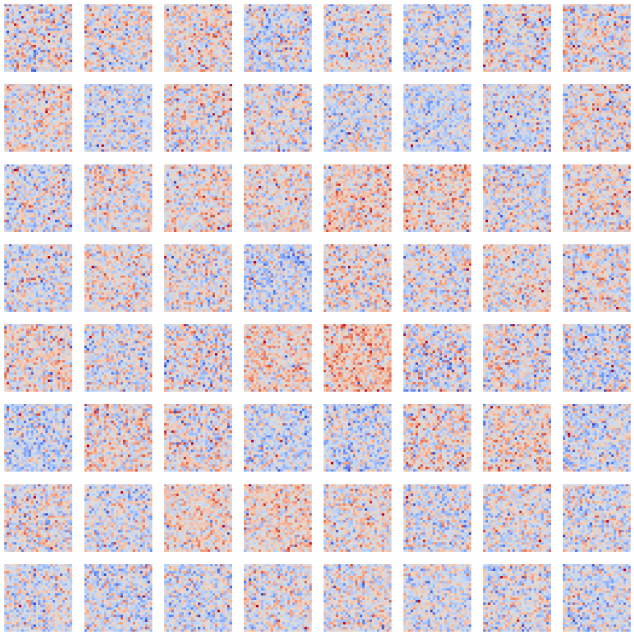
Figure 5: Confusion matrices of MLP and CNN.



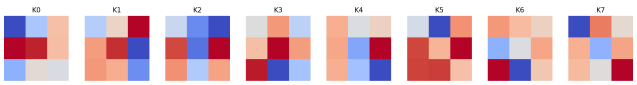
(a) MLP misclassified samples.

(b) CNN misclassified samples.

Figure 6: Examples of misclassified images.



(a) MLP first-layer weights.



(b) CNN first-layer kernels.

Figure 7: Visualization of learned parameters.